UGANDA CHRISTIAN UNIVERSI TY

A Center of Excellence in the Heart of Africa

CSC2107: Design and Analysis of Algorithms

Assignment 5.1

Naive vs. Memoized Fibonacci

GROUP 5 MEMBERS

MULINDWA EMMANUEL S25B23/059

BUHAMIZO ELIJAH K S25B23/064

WANDEBA GLEN JEREMOTH S25B23/105

ABENAITWE PAUL M25B23/058

MUGABI JEREMY S25B23/011

MWALE CHRISTIAN PAUL S25B23/036

Due: October 22, 2026

Part 3: Analysis

3.1 Recurrence relations

Algorithm A counts one call for every run of the function. For n = 0 or n = 1 it makes 1 call and stops. For any larger n it calls itself twice. If C_A(n) is the number of calls for input n, then:

$$C_A(n) = C_A(n - 1) + C_A(n - 2) + 1, \quad \text{with } C_A(0) = C_A(1) = 1$$

Algorithm B is the same shape, but when the table is empty the second recursive call always finds its answer already stored, so it costs one extra call on top of the chain that computes everything from scratch:

$$C_B(n) = C_B(n - 1) + 2, \quad \text{with } C_B(0) = C_B(1) = 1$$

3.2 Why Algorithm A needs 2 F(n+1) - 1 calls

Claim: C_A(n) = 2 * F(n + 1) - 1 for all n >= 0. We prove it by induction.

Base case: F(1) = 1 and F(2) = 1, so 2 * F(1) - 1 = 1 = C_A(0) and 2 * F(2) - 1 = 1 = C_A(1). Both hold.

Induction step. Assume the claim holds for n - 1 and n - 2. Then:

$$C_A(n) = C_A(n - 1) + C_A(n - 2) + 1$$

$$= (2 * F(n) - 1) + (2 * F(n - 1) - 1) + 1$$

$$= 2 * (F(n) + F(n - 1)) - 1 = 2 * F(n + 1) - 1$$

The claim holds, so the number of calls is 2 * F(n + 1) - 1.

Our data confirms it. At n = 35 the counter recorded 29,860,703 calls and F(36) = 14,930,352, and 2 * F(36) - 1 = 29,860,703 exactly.

Because F(n) grows like phi to the power n over sqrt(5), where phi = (1 + sqrt(5)) / 2 = 1.618, the number of calls grows like phi to the power n, so the running time is exponential, about O(phi^n). Since phi < 2 this is also bounded above by O(2^n), which is the common loose upper bound.

The measurements agree. Each small increase of n multiplies the call count and the time by about phi: from n = 34 to n = 35 the calls grew by the factor 29,860,703 / 18,454,929 = 1.618, which is phi itself.

3.3 Why Algorithm B needs 2n - 1 calls

With an empty table, fib is first called once for every value from n down to 1. Each of those fresh calls spawns two recursive calls, but the inner one for the smaller index is done computing the whole value chain, and the outer one returns straight from the table. Counting gives one fresh computation that costs n + ...

A cleaner way: C_B(n) = C_B(n - 1) + 2 with C_B(1) = 1. Unrolling gives 1 + 2 * (n - 1) = 2n - 1 calls. Our data matches: at n = 100 the counter showed 199 calls, and 2 * 100 - 1 = 199. At n = 500 it showed 999 calls, also exactly 2n - 1.

Time complexity of Algorithm B: O(n), it is linear. Space complexity: O(n) for the memoization table, plus O(n) for the recursion stack. In total O(n) space.

3.4 Complexity class of each algorithm

- Algorithm A is exponential. Its call count is 2 * F(n + 1) - 1, which grows like phi^n. The data backs this: every extra step of n multiplied the calls by about 1.6 and the time by about 1.6. Going from n = 20 to n = 35, only 1.75 times more input, multiplied the calls by 1,365 and took about 1,200 times longer to run.
- Algorithm B is linear. Its call count is 2n - 1 and its running time grows in step with n. The table confirms: 10 times more input (n = 50 to n = 500) produced 10 times more calls (99 to 999, exactly 2n - 1 each) and took 15.2 times longer on the timer, from 0.000026 s to 0.000394 s. Both are multiples of n, close to linear, and the small spread above 10 comes from constant work per call, which a linear algorithm always has.

3.5 Feasibility of Algorithm A at n = 1000

If Algorithm A were run with n = 1000 it would make 2 * F(1001) - 1 calls. F(1000) has 209 digits, and 2 * F(1001) - 1 is a number with 210 digits, about 1.4 x 10^209 calls.

Our timed run at n = 35 did 29,860,703 calls in 7.14 seconds, so the machine made about 4.2 million calls per second. At that rate 1.4 x 10^209 calls would need about 10^202 seconds, which is more than 10^195 years. The computer would die, the sun would die, and the answer still would not arrive. It is not feasible in any sense; it is impossibly far beyond feasible.

Part 4: Pushing to n = 1000

4.1 What happens with default settings

Running Algorithm B with n = 1000 under the default settings raises RecursionError: maximum recursion depth exceeded. The reason is that Python does not let a function call itself more than 1000 times deep by default; sys.getrecursionlimit() returns 1000, and Algorithm B needs about 1000 recursive frames just to reach F(0) before anything returns, plus a few extra frames already used by the program itself, so the depth cap is hit.

4.2 Fix 1: Iterative version, no recursion

Keep two variables, a and b, and repeat n times: a, b = b, a + b with a starting at 0. This computes F(1000) in a fraction of a millisecond with no recursion possible to blow up.

- Trade off: it never hits the recursion limit and uses constant extra memory, O(1) space.
- Trade off: it does not look like the pseudocode any more. It is a different algorithm, so it cannot be used to count calls of the original recursion, and for a learner it hides the recursive structure the exercise was teaching.

4.2 Fix 2: Raise the recursion limit

Call sys.setrecursionlimit(5000) before running Algorithm B. With the limit raised, Algorithm B computed F(1000) in under a millisecond, and its answer matched the iterative one (both give a 209 digit number).

- Trade off: it keeps the original recursive memoized structure, so it is easy to compare with Algorithm A and honest to the pseudocode.
- Trade off: it is dangerous. The Python recursion limit is a safety net that stops the program from asking the operating system for more stack than the stack really has. Raising it too far can cause a hard crash instead of a safe Python exception. It also slows the program down, since every recursive call costs real stack operations, while the iterative loop does not.

4.3 The subtle bug in the original pseudocode

Look at the line if fibtable[n]. The pseudocode only checks whether the key n is present. But there is one value of n for which this check fails even though the value was already stored: n = 0. F(0) = 0 is stored in the table, and an if that tests the number 0 acts as false, so the code recomputes F(0) every time instead of returning the stored 0.

Does it affect correctness? No. Recomputing F(0) always returns 0, so every answer is still right. Does it affect complexity? No. F(0) is recomputed at most once per outer run, so it adds at most one call, a constant. The complexity stays linear. The right safe test is if n in fibtable, which is what our Python implementation does.

Part 5: Conclusion

As n approaches 1000, Algorithm B is the only usable algorithm. Algorithm A is exponential. Its calls grow like 2 * F(n + 1) - 1, a number with 210 digits for n = 1000, so at the measured rate of 4.2 million calls per second it would need about 10^202 seconds. Algorithm B is linear. It makes 2n - 1 calls, finished F(1000) in under a millisecond in our run, and even the plain Python answer with memoization comes out in well under a second once the recursion limit is raised. No fixed factor states how much faster B is, because A cannot finish at all. At n = 35, where both still run, A needed 29,860,703 calls and 7.14 seconds while B needed 69 calls and under a tenth of a millisecond, already a speedup of the order of 100,000 times, and that gap grows without limit as n grows. The principle behind this is overlapping subproblems. Plain recursion solves the same small problems over and over and over again: F(n - 2) is computed inside two different branches, and every leaf recomputes F(1) and F(0). Memoization removes the waste by remembering each answer the first time it is computed and returning it from memory every later time it is asked for. When a problem breaks into subproblems that repeat, memory trading for time turns an exponential workload into a linear one.

Estimated Part 5 word count: 250 words (limit 300).

Appendix: Measurements

A.1 Results table

Execution time was measured with time.perf_counter(). The run of Algorithm A was stopped well before any run reached 60 seconds; the largest n measured for Algorithm A is 35, as listed in the assignment. Each Algorithm B run cleared the table first. A value of -1 in the calls column means the run stopped with a RecursionError, which Part 4 explains.

Algorithm	n	calls	time (s)
A	5	15	0.000004
A	10	177	0.000032
A	15	1,973	0.000394
A	20	21,891	0.006013
A	25	242,785	0.060900
A	30	2,692,537	0.626198
A	35	29,860,703	7.141777
B	10	19	0.000010
B	50	99	0.000026
B	100	199	0.000050
B	200	399	0.000153
B	500	999	0.000394
B	1000	-1 (RecursionError)	0.000588

A.2 Plots

Figure 1: n plotted against the number of function calls for both algorithms. The y axis is drawn on a log scale because Algorithm A grows exponentially and spans many orders of magnitude. On a plain linear axis, its curve would rise to millions while every point of Algorithm B would be squashed flat against the bottom and impossible to read. The log scale lets both lines be seen on one chart.

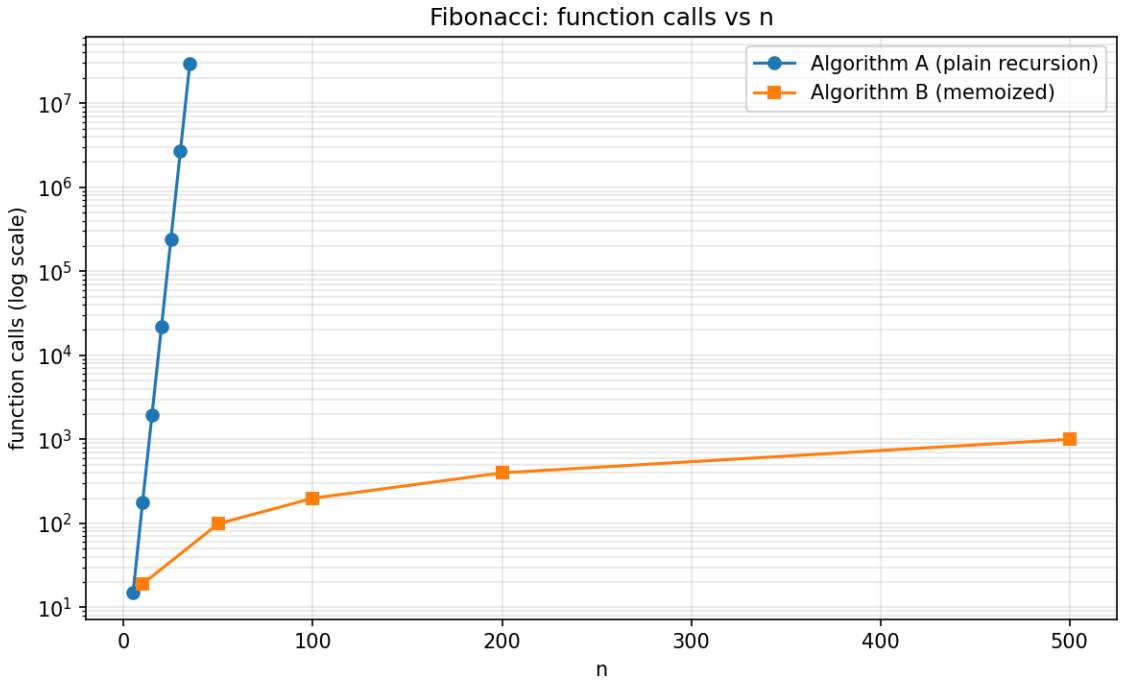


Figure 2: n plotted against execution time, also on a log scale for the same reason.

